

데이터 분석 입문반

한 기 영

(주)데이터인사이트 대표

지난 시간 복습

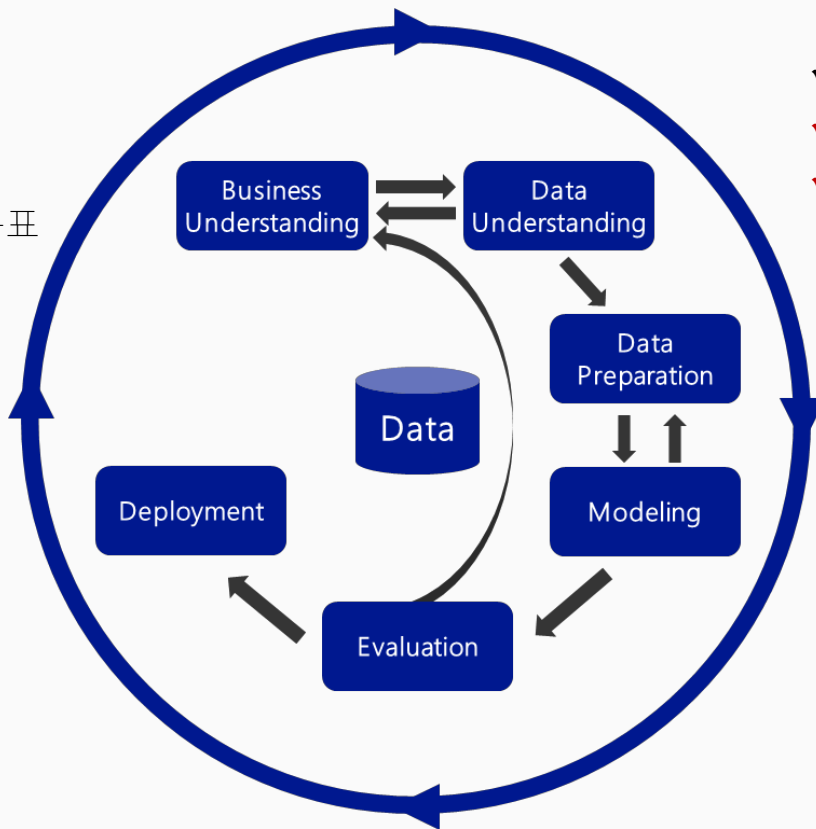
전체 Process(CRISP-DM)

무엇이 문제인가?

- ✓ 비즈니스 문제정의
- ✓ 데이터분석 방향, 목표
- ✓ 초기 가설 수립

$x \rightarrow y$

- ✓ 모델 관리
- ✓ AI 서비스 구축



- ✓ 원본식별
- ✓ 분석을 위한 구조 만들기
- ✓ 데이터분석 EDA & CDA

- ✓ 모델링을 위한 데이터 구조 만들기
 - 모든 셀은 값이 있어야 한다.
 - 모든 값은 숫자 여야 한다.
 - (필요 시) 숫자의 범위가 일치

- ✓ 모델을 만들고
- ✓ 검증하기

문제가 해결 되었는가?

- ✓ 기술적 관점 평가
- ✓ 비즈니스 관점 평가

분석할 수 있는 데이터 구조

✓ 기본 0 | 2차원

- Table, 2차원 Array, DataFrame

① 열, 정보, 변수, 요인

Feature, X, input, (독립변수)

Target, y, Output,
Label, (종속변수)

② 행,
분석단위,
샘플,
관측치,
Data point

[illegible]

분석할 수 있는 데이터 구조



NumPy

1D Array

3	2
---	---

2D Array

1	0	1
3	4	1

3D Array

1	7	9
5	9	3
7	9	9

수치 연산



pandas

	Name	Code-Name	Age	Weight in kgs
0	Amar	Alpha	23	69
1	Barsha	Bravo	25	54
2	Carlos	Charlie	22	73
3	Tanmay	Tango	27	70
4	Misbah	Mike	29	74

비즈니스 데이터 표현

2. 파이썬 및 판다스 복습

데이터 분석에 꼭 필요한 내용만 다룹니다.

파이썬 기본 자료형

✓ 리스트와 딕셔너리는 익숙하게 다룰 수 있어야 합니다.

methods : 방식-

	의미	만들기	조회	값 추가
리스트 List	✓ 여러 값들로 구성된 목록 ✓ 값에는 다양한 자료형이 올 수 있음	<code>a = [2, 5, 1, 3, 7]</code>	인덱스로 조회 <code>a[2:4]</code>	<code>a.append(10)</code>
딕셔너리 Dictionary	✓ <code>key:value</code> 로 구성 ✓ value에는 다양한 자료형이 올 수 있음	<code>b = {'k1': 2, 'k2': [1, 2, 3]}</code> <i>value로 List 가능</i>	key로 조회 <code>b['k1']</code>	<code>b['k3'] = 1.72</code>

* display() ~ print() 와 비슷한 함수

Range (시작, 끝, 간격) → Range (1, 12, 2) → 1 3 5 7 9 11
→ 생각가능
→ 무조건

List

List.append (요소)

Dictionary (딕셔너리)

↳ key

↳ value

⇒ dict_a = {'v1': 32, 'v2': [1, 2, 3], 'v3': {'a': 1, 'b': 2}}

dict_a ['v1']

" ['v3'] → {'a': 1, 'b': 2}

시리즈(Series)와 데이터프레임(Dataframe)

- ✓ 데이터프레임 : 2차원 구조 $\rightarrow$ 표 형태
- ✓ 시리즈 : 1차원 구조
 - 데이터프레임에서 열 하나를 띄어 내면 시리즈!

시리즈 $\subseteq$ 데이터프레임

★

	day	smoker	total_bill	tip	size
0	Fri	No	73.68	11.25	9
1	Fri	Yes	252.20	40.71	31
2	Sat	No	884.78	139.63	115
3	Sat	Yes	893.62	120.77	104
4	Sun	No	1168.88	180.57	167
5	Sun	Yes	458.28	66.82	49
6	Thur	No	770.09	120.32	112
7	Thur	Yes	326.24	51.51	40

시리즈

데이터 프레임

CSV파일에서 데이터 읽어 오기

- ✓ 보통 CSV 파일을 읽어와 데이터프레임 생성
- ✓ `pd.read_csv(파일경로 및 이름)`

```
# 데이터 읽어오기
path = 'https://raw.githubusercontent.com/DA4BAM/dataset/master/population_sample.csv'
pop = pd.read_csv(path)

# 상위 10행만 확인
pop.head(10)
```

$data.head() \leftrightarrow data.tail()$

data.shape
↳ 행과 열 수

	year	household	k_male	k_female	f_male	f_female
0	1981	NaN	4160	4191	NaN	NaN
1	1982	NaN	4160	4191	NaN	NaN
2	1983	2116.0	4160	4191	NaN	NaN
3	1984	2246.0	4160	4191	NaN	NaN
4	1985	2338.0	4788	4838	7.0	6.0
5	1986	2428.0	4899			

data['열 이름'] 에 시리즈

* data['열 이름'].unique() → 고유값만 파악
" . value_counts() → 값의 횟수
" . sum() → 열 합계 조회
" . max() → 열의 최댓값 조회

[데이터] 프레임 조회

→ data['Attrition'] / data.~~Attrition~~
원장

data[['Attrition', 'Age']]
리스트 → 2개의 시리즈 추출

조건으로 조회 → 결과는 True / False로 출력.

⇒ data['Distance From Home'] > 10

⇒ data.loc[data['Distance From Home'] > 10] → 10보다 큰 값만 조회

* : dictionary 과 json 의 형식이 같다.

Pd.read_csv() / .head() 상위 5개

data.shape → 행 수 · 열 수 확인

data.columns → column의 이름
↳ print() 사용

data['column이름'] → column만 추출

조건으로 조회 : .loc[행 조건 , 열 이름]

✓ 행 조건

- 조건문 형태 → 조건에 만족하는 데이터만 조회

✓ 열 이름

- 생략 가능
- 열 이름 1개 → 결과 : 시리즈
- 열 이름(여러 개)을 **리스트**로 → 결과 : 데이터프레임

```
# total_bill (열 값이 6.0) 보다 큰 행 조회  
tip.loc[tip['total_bill'] > 6.0]
```

	total_bill	tip	sex	smoker	day	time	size
23	39.42	7.58	Male	No	Sat	Dinner	4
59	48.27	6.73	Male	No	Sat	Dinner	4
141	34.30	6.70	Male	No	Thur	Lunch	6
170	50.81	10.00	Male	Yes	Sat	Dinner	3
183	23.17	6.50	Male	Yes	Sun	Dinner	4
242	48.33	9.00	Male	No	Sat	Dinner	4

조건으로 조회 : .loc[행 조건 , 열 이름]

✓ 여러 조건을 만족하는 행 조회

- ~~and~~ 대신 ~~&~~, ~~or~~ 대신 ~~|~~ 연산자를 사용해야 하며, 각 조건은 소괄호로 묶어야 함

and로 여러 조건 연결

```
tip.loc[(tip['tip'] > 6.0) & (tip['day'] == 'Sat')]
```

or로 여러 조건 연결

```
tip.loc[(tip['tip'] > 6.0) | (tip['day'] == 'Sat')]
```

✓ 만약 열 이름만 지정하고 모든 행을 가져오려면...

모든 행 :

```
tip.loc[ : , ['tip', 'size'] ]
```

조건으로 조회 : .loc[행 조건 , 열 이름]

✓ 편리한 isin(), between() 메소드 *※ 여러개의 경우 리스트를 활용한다.*

- isin([값1, 값2,..., 값n]): 값1 또는 값2 또는...값n인 데이터만 조회

```
tip.loc[tip['day'].isin(['Sat', 'Sun'])]
```

- between(값1, 값2): 값1 ~ 값2까지 범위안의 데이터만 조회

```
tip.loc[tip['size'].between(1, 3)]
```

dataframe.loc [조건, ['열이름 1', '열이름 2']]

↳ 조건을 충족시키는 열이름 1, 열이름 2